

Model Compression and Knowledge distillation

Model Compression

url: https://www.researchgate.net/publication/221653840_Model_compression

Authors: Cristian Bucila, Rich Caruana, Alexandru Niculescu-Mizil

Date of publication: August 2006

Problem statement:

Ensemble or large complex models are often a high performing **generalized** model, however such models are large in size which require high computation power and have latency in inference time. Which make them inapplicable for real-time inference or running on computationally less powerful devices like edge or CPU devices.

This paper addressed that problem by proposing a model compression technique to use a fast and compact model to approximate the function learned by a slower, larger, but better performing model. In a simple word, training a small less complex model to achieve the similar performance of the ensemble model.

Objective:

Training a fast compact model to achieve the similar performance of the slow large model using synthetic data technique. Unlike the true function that is unknown, the function learned by a high performing model is available and can be used to label large amounts of pseudo data.

A fast, compact and expressive model trained on enough pseudo data will not overfit and will approximate well the function learned by the high performing model.

Methodology:

Initially a slow large model is trained on an available dataset, after that slow large model is used to pseudo label the available unlabeled pseudo large dataset. Once a pseudo large dataset is ready, a small compact model is trained on a pseudo large dataset to achieve the similar performance of the slow large model.

For some domains like text, images, it is easy to get large amounts of unlabeled data from the web and use it as a pseudo dataset. However, for other domains, unlabeled dataset are not readily available, so have to generate synthetic data. Please note that the synthetic data must have the same distribution of the original training data.

Authors have proposed three methods to generate pseudo data:

- **RANDOM**

- Generate pseudo-inputs by sampling features **independently at random** from their observed marginal distributions.
- (**Marginal distribution** = the distribution of a single feature/variable *ignoring the others*. **Observed** = estimated directly from the training data you already have.)
- Example: if feature x_1 is height and x_2 is weight, pick a random height from the height distribution and a random weight from the weight distribution, without worrying if that combo makes sense.
- **Pro:** Simple and covers a large input space.
- **Con:** Can create unrealistic feature combinations (e.g., 5-year-old with 200 lbs).

- **NBE (Naive Bayes Estimator)**

- Uses a Naive Bayes model trained on the original data to generate new samples.
- Because Naive Bayes assumes conditional independence, it samples features conditioned on class, giving more *plausible* combinations than **RANDOM**.
- **Pro:** Generates more realistic examples than RANDOM, and can handle both discrete and continuous attributes
- **Con:** Limited by Naive Bayes' strong independence assumption.

- **MUNGE**

Input: Original training set.

Step 1 – Find Neighbors:

- For each example e , identify its closest neighbor e' .
- Distance metric:
 - Euclidean distance $\rightarrow$ continuous attributes (scaled to $[0, 1]$).
 - Hamming distance $\rightarrow$ nominal attributes.

Step 2 – Attribute Modification:

- Nominal attributes:
 - Swap values between e and e' with probability p .
 - Leave unchanged with probability $1 - p$.
- Continuous attributes:
 - With probability p :
 - Replace e_a with a random value $\sim \text{Normal}(\text{mean} = e'_a, \text{std} = |e_a - e'_a|/s)$.
 - Replace e'_a with a random value $\sim \text{Normal}(\text{mean} = e_a, \text{std} = \text{same as above})$.

Output: New artificial data set where values are swapped or perturbed between neighboring cases.

Pros: works with both continuous and nominal attributes, and preserves the distribution of the original dataset to the synthetic dataset

Cons: less effective with small and noisy datasets

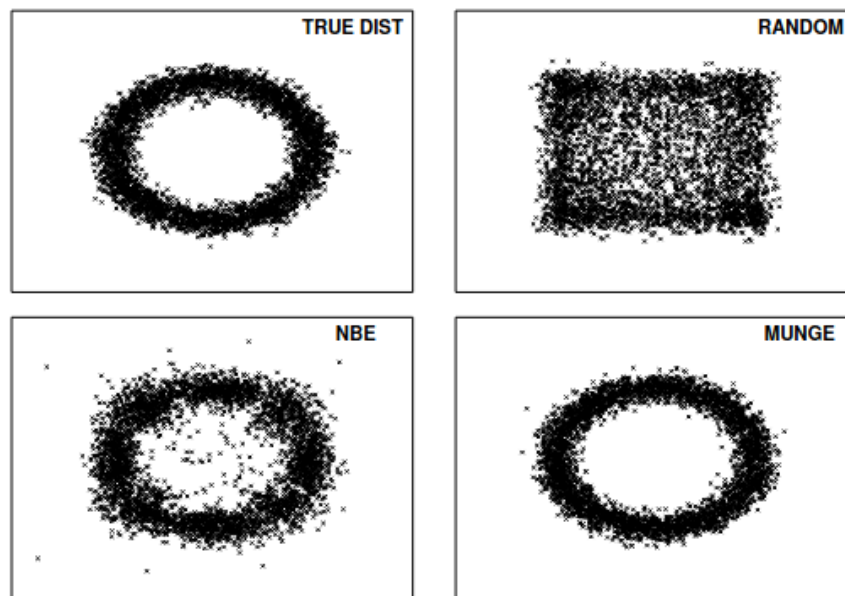


Figure 1: Synthetic data generated for a simple 2D problem.

Findings:

- The models trained with ensemble selection perform considerably better than the neural net models trained on the same training datasets

- Figure 2 shows the performance of different model trained with different size of pseudo dataset

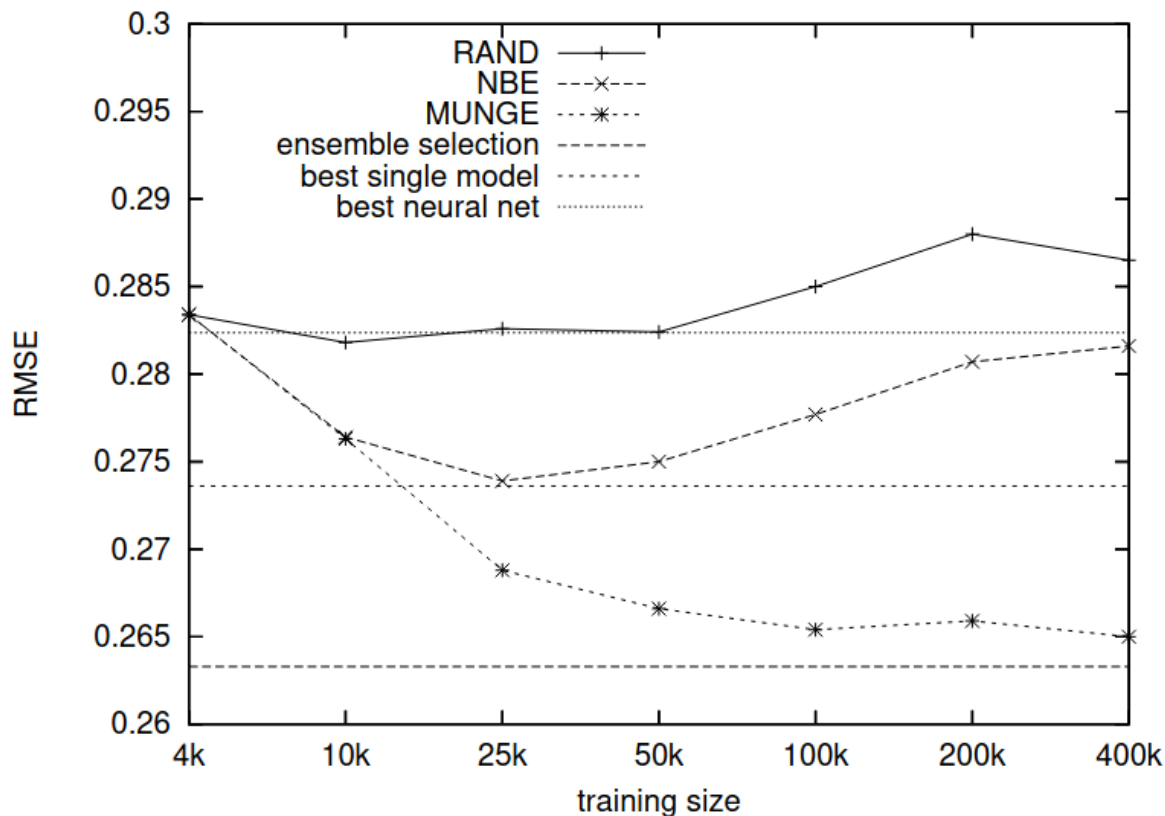


Figure 2: Average perf. over the eight problems.

- On the HS problem, the performance of the mimic neural net trained on MUNGE pseudo data is better than the performance of the ensemble model it is trying to mimic. It could be because ensemble models overfit the data whereas compressed models are more generalized.

Conclusion

- Large ensemble models are accurate but too slow and resource-heavy.
- Model compression trains small, fast neural nets to mimic large ensemble models.
- Compression works by labeling unlabeled data with the big model and retraining.
- When unlabeled data is missing, **MUNGE** generates pseudo-data (though less effective).
- Experiments show compressed nets are nearly as accurate, but **1000× smaller and faster**.

Distilling the Knowledge in a Neural Network

url: <https://arxiv.org/abs/1503.02531>

Authors: Geoffrey Hinton, Oriol Vinyals, Jeff Dean

Date of Publications: 9 Mar 2015

Problem statement:

One of the simplest ways to improve the performance of any ML model is by training multiple models on the same data and averaging their results. However, this ensemble technique is too computationally expensive.

The authors have proposed a new approach for training a small and compact model whose performance is going to be similar to the ensemble models.

Objective:

Once the cumbersome model has been trained, then use a different kind of training, which is called “distillation” to transfer the knowledge from the cumbersome model to a small model that is more suitable for deployment.

Keypoints:

- When training a classification model, the objective is to maximize the average log probability of the correct class but the side effect is the model assigns the probability to all the other classes.
- The relative probability of the incorrect classes tell us a lot about how the cumbersome model tends to generalize.
- For instance, an image of BMW might have
 - $P_{\text{bmw}} = 0.7$ (probability of being bmw class)
 - $P_{\text{truck}} = 0.25$ (probability of being truck)
 - $P_{\text{carrot}} = 0.05$ (probability of benign carrot)

Here, the model has a small chance of being mistaken for a truck, but that mistake is still many times more probable than mistaking it for a carrot. This basically means the model is able to find the similarity between the closest classes which make them more generalized.

- A small model trained to generalize in the similar way as the cumbersome model will do much better on train data than a small model trained in a normal way on the same training set.
- One way of transferring the generalization ability of the cumbersome model to a small model is to use the class probability produced by the cumbersome model as a soft target for training a small model.
 - **Soft target:**

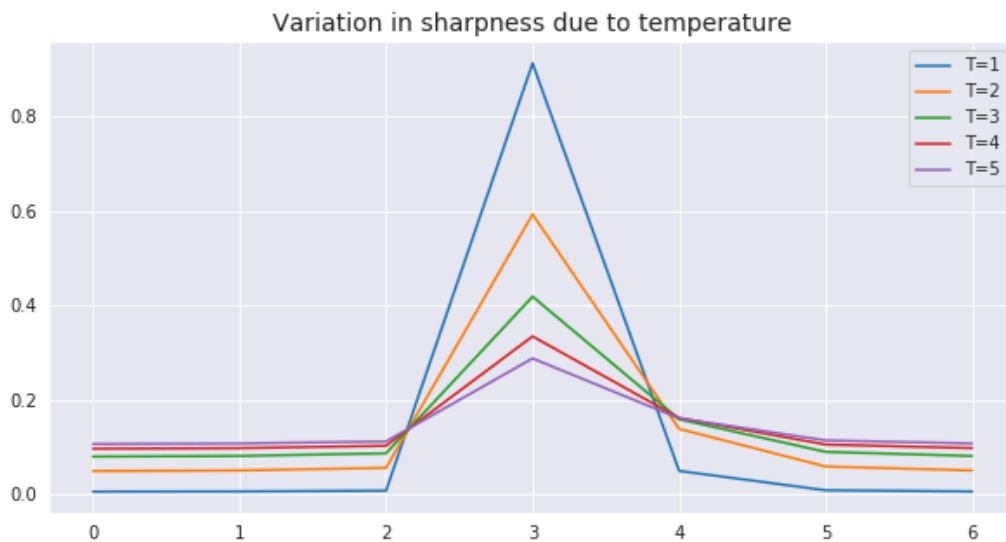
- The probability distribution output by the teacher/cumbersome model (e.g., $[0.05, 0.85, 0.10]$).
 - This rich signal shows the teacher's confidence for all classes, including incorrect ones, for instance, it reveals that it is not only the image of BMW but also that the teacher model thinks it is a bit like a truck, and is the key to knowledge distillation.
- **Hard target:**
 - The standard, one-hot encoded correct label (e.g., $[0, 1, 0]$). It provides a definite answer.
- For transfer stage, we could use the same training set or a separate transfer set (could be completely unlabeled or labeled)
 - When the cumbersome model is a large ensemble of simpler models, we can use an arithmetic or geometric mean of their individual predictive distributions as the soft targets
 - Soft targets with high entropy carry more information per example and produce smoother (lower-variance) gradients, allowing the small model to train faster, with less data, and at a higher learning rate than when using hard targets.
 - For tasks like MNIST, the probability of similar classes are near to zero thus does not cause much contribution in training, one way to address this problem is using logits instead of probability distribution of prediction and use maximize the square difference between the teacher logits and student logits.
 - The author of this paper addressed this issue through the concept of “**distillation**” where they introduce a parameter “**temperature**” in the softmax function, where this temperature value could be raised until the teacher produces a suitable soft set of targets.

Methodologies

Softmax is used to convert the output of the last layer i.e logits $\mathbf{z}$ into class probabilities $\mathbf{q}$

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

where , T is a hyperparameter called Temperature. Higher the value softer probability distribution over classes



When $T=1$ the probability of class 3 is more than 0.8 and other classes are less than 0.1, on the other hand when $T=5$, we can see that the probability of class 3 is dropped into approx. 0.3 where other class probability has increased comparatively.

1. Simplest form of distillation

- Train a teacher model
- product soft targets by setting high temperature value for the given transfer set
- Train the student model by setting the same high temperature with the generated soft targets
- During inference of the student model we use the temperature = 1

Cross entropy loss for distillation:

$$\mathcal{L}_{\text{KD}} = - \sum_{i=1}^C p_i^T \log q_i^S$$

Where.

- P^T is the soft targets from teacher model,
- q^S is the student prediction

2. Weighted objective function

- When the transfer set contains all or some labeled example then we can use a weighted objective function to improve the performance
 - The first objective function is the cross entropy with the hard target (correct labels). Here, softmax function of student model is computed with $T=1$

- ii. The Second objective function is the cross entropy with soft targets (as mentioned in the above approach)
- b. Setting lower weights to the first objective function tends to give a better performance result

$$\mathcal{L} = \alpha \mathcal{L}_{\text{CE}}(q^S, y) + (1 - \alpha) T^2 \mathcal{L}_{\text{KD}}(q^S, p^T)$$

- $\alpha \in [0, 1]$ balances hard labels and teacher knowledge.

- c. Since the magnitudes of the gradients produced by the soft targets scale as $1/T^2$ it is important to multiply them by T^2 when using both hard and soft targets

3. Special case of distillation (matching logits)

Each case in the transfer set contributes a cross-entropy gradient, dC/dz_i , with respect to each logit, z_i of the distilled model. If the cumbersome model has logits v_i which produce soft target probabilities p_i and the transfer training is done at a temperature of T , this gradient is given by:

$$\frac{\partial C}{\partial z_i} = \frac{1}{T} (q_i - p_i) = \frac{1}{T} \left(\frac{e^{z_i/T}}{\sum_j e^{z_j/T}} - \frac{e^{v_i/T}}{\sum_j e^{v_j/T}} \right) \quad (2)$$

- in the high temperature limit, distillation is equivalent to minimizing $1/2(z_i - v_i)^2$

Findings

- distillation lets the small net inherit the large net's "translation robustness" without ever being explicitly trained on translated data.
- Knowledge distillation allows a student to generalize to classes it hasn't seen, by inheriting relational knowledge from the teacher, but the class bias must be corrected for good accuracy.
- Distilling an ensemble of DNN acoustic models into a single model yields significantly better performance than training a model of the same size directly on the data.